

DATA & AI PORTFOLIO · PROJECT NOTEBOOK 06

Real-Time Transaction Streaming & Fraud Detection

Streams, sliding windows, and the recall ceiling a rule set cannot cross

Author Nikhil Sinha

Repository github.com/Nikhil201716/06-Realtime-Transaction-Streaming-Pipeline

Stack Redis Streams · Python · DuckDB · Ollama (qwen2.5:0.5b) · Streamlit

Edition 1.0

How to read this book

This book is written to be read from the beginning by someone who has not seen the project and does not yet know the techniques it uses. It is not API documentation - it is closer to a course text built around one real working system.

Two conventions run throughout. Callouts marked **WHY** explain a design decision and name the alternative that was rejected; callouts marked **WARNING** mark a trap that is easy to fall into and hard to notice afterwards. There is a chapter of pure terminal commands so the project can be reproduced without reading anything else, and a chapter of exercises with worked solutions.

Every code listing is extracted from the repository at the moment this book is built, so nothing here can drift out of step with the code it describes. If a listing looks wrong, the code is wrong.

All data in this project is synthetic. No real customer, transaction, or company data appears anywhere.

Contents

1. The Business Problem

- 1.1 The decision has to happen before the money moves
- 1.2 Two ways to be wrong, and they are not symmetric
- 1.3 What was built and measured

2. Streams, Logs and Consumer Groups

- 2.1 A queue forgets; a log remembers
- 2.2 XADD, XREADGROUP, XACK
- 2.3 At-least-once, and what it obliges you to do
- 2.4 The loop that ties it together
- 2.5 The producer and the label that scoring never sees

3. Scoring Rules, and the Per-Account Baseline

- 3.1 The four rules
- 3.2 Why the z-score uses the account's own baseline
- 3.3 Additive scoring, and its two properties

4. The Sliding Window

- 4.1 The question a velocity rule asks
- 4.2 The sorted-set pattern
- 4.3 Why a pipeline, and why the order matters
- 4.4 This is processing-time windowing, and that is a limitation

5. A Guided Tour of the Codebase

- 5.1 How to read this tour
- 5.2 `scripts/generate_reference_data.py`
- 5.3 `scripts/producer.py`
- 5.4 `pipeline/redis_client.py`
- 5.5 `pipeline/consumer.py`
- 5.6 `pipeline/scoring_rules.py`
- 5.7 `pipeline/explain.py`

5.8 `scripts/evaluate_detection.py`

5.9 `dashboard/streamlit_app.py`

6. Evaluating a Detector Honestly

6.1 The four outcomes

6.2 What makes this evaluation trustworthy

6.3 What these numbers do not establish

7. The Recall Ceiling

7.1 Where the misses are

7.2 Why card testing is nearly invisible

7.3 The warm-up, derived

7.4 What an attacker does with this knowledge

7.5 The false positive nobody can distinguish

8. Measuring Latency Honestly

8.1 Measure end to end, not stage by stage

8.2 The measured distribution

8.3 What these numbers do not include

9. The Explanation Layer

9.1 The division of labour

9.2 Why this matters beyond hallucination

10. Worked Examples

10.1 Example 1 - Scoring a stolen-card transaction

10.2 Example 2 - Scoring a card-testing transaction

10.3 Example 3 - The recall ceiling, derived

10.4 Example 4 - Overall recall is at its structural maximum

11. Running the Project, Step by Step

11.1 Prerequisites

11.2 Step 1 - Clone and enter the repository

11.3 Step 2 - Create an isolated environment

11.4 Step 3 - Install dependencies

- 11.5 Step 4 - Run the pipeline, stage by stage
- 11.6 Run producer and consumer together
- 11.7 Generate an explanation for a flagged transaction
- 11.8 Inspect the stream directly
- 11.9 Inspect a velocity window
- 11.10 Query the scored warehouse
- 11.11 Launch the operations console
- 11.12 Command reference card
- 11.13 Troubleshooting

12. The Live Dashboard

- 12.1 Reading a stream that has already been scored
- 12.2 Why the reasons column carries the product

13. Results

- 13.1 The numbers
- 13.2 The four findings worth defending
- 13.3 The honest tooling note

14. Exercises, with Solutions

- 14.1 Exercises
- 14.2 Solutions

15. Appendix

- 15.1 A - Repository map
- 15.2 B - Glossary
- 15.3 C - Reproducibility
- 15.4 D - Where this leads next

CHAPTER 1

The Business Problem

Why fraud detection cannot be a batch job, what the decision actually is, and the cost of being wrong in each direction.

1.1 The decision has to happen before the money moves

Every project so far in this portfolio has been a batch. Data arrives, a pipeline runs, a report appears. Nothing breaks if the answer is a few hours old.

Fraud detection has no such luxury. The card is being presented right now and the authorisation decision has a few hundred milliseconds. A nightly job that identifies yesterday's fraud is not a detector - it is an accounting exercise, because the money has already gone.

THIS IS WHAT MAKES STREAMING A DIFFERENT DISCIPLINE, NOT A FASTER BATCH

You cannot see the whole dataset. There is no `GROUP BY` over all history, no second pass, no opportunity to sort. Every decision is made on one event plus whatever state you chose to keep in advance.

That constraint reshapes everything: the features must be computable incrementally, the state must be bounded, and the answer must be produced before the next event arrives. Chapter 4's sliding window exists entirely because 'count this account's recent transactions' has to be answered without querying anything.

1.2 Two ways to be wrong, and they are not symmetric

1.1 The asymmetric costs of the two errors

	Miss a fraud (false negative)	Flag a legitimate customer (false positive)
Immediate cost	the money is gone	a declined card
Who absorbs it	the bank	the customer, publicly
Visibility	shows up in a chargeback weeks later	the customer notices instantly
Second-order cost	regulatory attention	the customer stops using the card

This is why fraud systems report **precision and recall** separately rather than accuracy. With 44 fraud events among 415, a detector that flags nothing scores 89% accuracy - and is worthless.

ACCURACY IS THE WRONG METRIC WHENEVER ONE CLASS IS RARE

The base rate here is 10.6%. In real card fraud it is closer to 0.1%, where a do-nothing detector scores 99.9% accuracy. Any evaluation reporting accuracy on an imbalanced problem is either uninformed or hiding something.

The same point recurs in Project 2's churn model and Project 15's fraud work. It is worth internalising once: **on rare events, accuracy measures the base rate, not the model.**

1.3 What was built and measured

1.2 Measured results

Metric	Value
Events streamed and scored	415
Fraud events (injected)	44
Precision	97.2%
Recall	79.5%
F1	0.875
Confusion matrix	TP 35 / FP 1 / FN 9 / TN 370
Latency, average / p95 / max	6.2 ms / 21.1 ms / 30.4 ms

BOTH INTERESTING NUMBERS ARE STRUCTURAL, NOT TUNED

The 79.5% recall is exactly the maximum this rule set can achieve on this data - all nine misses are events no rule could have fired on. And the single false positive is *score-identical* to fourteen of the true positives.

Neither fact is visible from the headline metrics, and both are derived in Chapter 7. They are the most useful things this project has to say.

CHAPTER 2

Streams, Logs and Consumer Groups

Why a stream is a log rather than a queue, what a consumer group buys, and why at-least-once delivery makes idempotency mandatory.

2.1 A queue forgets; a log remembers

The distinction decides what your system can do.

2.1 Queue semantics versus log semantics

	Queue	Log (stream)
On consume	the message is gone	the entry stays
Consumers	each message goes to one	many, each with its own position
Replay	impossible	read from any offset
New consumer	sees only future messages	can read all history
Examples	RabbitMQ, SQS	Kafka, Redis Streams

REPLAY IS THE PROPERTY THAT MATTERS HERE

A fraud detector's rules change constantly. With a log you can replay last week's traffic against a new rule set and measure what would have happened. With a queue that data is gone, and the only way to evaluate a rule change is to ship it.

It also makes development sane: the consumer can be stopped, fixed and restarted from the beginning, which is exactly how the runbook in Chapter 11 works.

2.2 XADD, XREADGROUP, XACK

Redis Streams provides three operations that carry the whole model.

1. **XADD** - append an entry. The producer's only job.
2. **XREADGROUP** - read entries as a member of a named consumer group. Redis tracks which entries have been delivered to which consumer and not yet acknowledged.
3. **XACK** - confirm an entry is fully processed. Until then it stays in the **pending entries list**.

Creating the consumer group

pipeline/consumer.py

```
def ensure_consumer_group(client: redis.Redis):
    try:
        client.xgroup_create(STREAM_RAW, CONSUMER_GROUP, id="0", mkstream=True)
        print(f"Created consumer group '{CONSUMER_GROUP}' on stream '{STREAM_RAW}'")
    except redis.exceptions.ResponseError as e:
        if "BUSYGROUP" in str(e):
            print(f"Consumer group '{CONSUMER_GROUP}' already exists - resuming.")
        else:
            raise

def update_velocity(client: redis.Redis, account_id: str, ts: float) -> int:
    key = f"{VELOCITY_KEY_PREFIX}{account_id}"
    pipe = client.pipeline()
    pipe.zadd(key, {f"{ts}:{account_id}": ts})
```

WHY A CONSUMER GROUP RATHER THAN A PLAIN READ

A plain `XREAD` gives every reader every message and tracks nothing. A consumer group gives you three things a real system needs: **work is divided** across consumers so you can scale horizontally; **position is tracked server-side** so a restart resumes rather than reprocessing everything; and **unacknowledged work is visible** in the pending list, so a consumer that dies mid-message does not silently lose it.

The project's own docstring makes the point: this is a consumer group, not a naive poll-and-delete loop - which is the shape that looks like streaming and loses messages the first time a process crashes.

2.3 At-least-once, and what it obliges you to do

Acknowledging *after* processing gives **at-least-once** delivery: a message is never lost, and may be delivered more than once if the consumer dies between doing the work and acknowledging it.

2.2 Delivery guarantees

Guarantee	Achieved by	Failure mode
At-most-once	ack before processing	messages silently lost
At-least-once	ack after processing	duplicates possible
Exactly-once	at-least-once + idempotent sink	requires design

EXACTLY-ONCE IS A PROPERTY OF THE SINK, NOT OF THE TRANSPORT

No transport can give you exactly-once by itself - the acknowledgement can always be lost after the work is done. What you build instead is at-least-once delivery into a sink where processing the same message twice has the same effect as processing it once.

That is Project 4's idempotency argument, arrived at from a different direction: there it was an upsert on a primary key so a rerun could not duplicate rows. Here it would be an upsert on `transaction_id`. **Project 14** implements this properly with a dedup window and measures it.

The consumer in this project acknowledges after the write, so it is at-least-once. On a duplicate delivery it would score the transaction again - which would also add a second entry to the velocity window, inflating the count for that account. That is an honest gap, and it is the second exercise in Chapter 14.

2.4 The loop that ties it together

Everything above meets in the consumer's main loop: read a batch from the group, update the window, score, measure latency, write, acknowledge.

The consumer loop

pipeline/consumer.py

```
resp = client.xreadgroup(CONSUMER_GROUP, CONSUMER_NAME,
```

TWO DETAILS IN THAT LOOP ARE WORTH PAUSING ON

The acknowledgement is the last statement. Everything - the window update, the scoring, the warehouse write - happens before `XACK`. That ordering is what makes the delivery at-least-once rather than at-most-once, and it is a deliberate choice to prefer duplicates over loss.

There is a batch size. `XREADGROUP` returns several entries per call rather than one, because a round-trip per event would dominate the 6.2 ms latency budget. Batching is the single largest lever on streaming throughput, and it costs a little latency at the head of each batch - the classic throughput-latency trade.

2.5 The producer and the label that scoring never sees

The producer emits normal traffic plus three fraud scenarios, and every event carries a `scenario_label`.

2.3 The three injected fraud scenarios

Scenario	Pattern	Events
<code>card_testing</code>	5-8 tiny charges seconds apart, home country, usual device	23
<code>stolen_card_spree</code>	3-5 large charges, foreign country, unrecognised device	15
<code>account_takeover</code>	device change then escalating amounts, home country	6

THE LABEL IS THE GROUND TRUTH AND THE SCORING CODE MUST NEVER SEE IT

`scoring_rules.py` receives only what a real system has at transaction time: amount, country, device, category, timestamp, and the account's historical profile. It never reads `scenario_label`, which exists purely so `evaluate_detection.py` can grade it afterwards.

This separation is the entire basis for trusting the evaluation. A detector with access to the labels would score perfectly and prove nothing - and the leak would be a single field name, easy to introduce and invisible in the results.

Notice also that the three scenarios were designed to exercise different rules. `stolen_card_spree` trips amount, country and device simultaneously. `account_takeover` deliberately stays in the home country so the obvious geography flag does not fire. `card_testing` trips nothing except velocity - which is what makes it the hard case and the subject of Chapter 7.

The card-testing generator - every field chosen to evade a rule

`scripts/producer.py`

```
def card_testing_events(account, rng):
```

READ THAT LISTING AS AN ATTACK DESIGN, NOT A DATA GENERATOR

Every field is a decision to avoid a signal. The amount is drawn from \$1-\$5, below the account's norm rather than above it. The country is the account's own. The device is the account's own. Only the timing - 5 to 8 events in under a second each - is anomalous.

An attacker probing a stolen card number has exactly these incentives: stay small so the charge is not noticed, stay familiar so no geography or device rule fires, and move fast because the card may be cancelled at any moment. The scenario is realistic *because* it is adversarial, and that is what makes it the one the rule set struggles with.

CHAPTER 3

Scoring Rules, and the Per-Account Baseline

Four rules, additive points, and why the amount rule compares an account against itself rather than against the population.

3.1 The four rules

The complete scoring logic

pipeline/scoring_rules.py

```

def score_transaction(event: dict, account: dict, recent_count: int) -> dict:
    """
    event: dict with amount, country, device_id, category, account_id
    account: dict with avg_transaction_amount, std_transaction_amount,
               home_country, primary_device_id
    recent_count: number of transactions from this account in the last
                  60 seconds (INCLUDING this one), from the Redis
                  sliding-window sorted set - see consumer.py
    """
    score = 0
    reasons = []

    # Rule 1: velocity
    if recent_count >= 4:
        score += 35
        reasons.append(
            f"High velocity: {recent_count} transactions from this account in the last 60 seconds "
            f"(a normal account rarely exceeds 2-3).")

    # Rule 2: amount anomaly (z-score vs. this account's own historical baseline)
    std = max(account["std_transaction_amount"], 1.0)
    z = (event["amount"] - account["avg_transaction_amount"]) / std
    if z > 6:
        score += 40
        reasons.append(
            f"Severe amount anomaly: ${event['amount']:.2f} is {z:.1f} standard deviations above "
            f"this account's typical ${account['avg_transaction_amount']:.2f} transaction.")
    elif z > 3:
        score += 20
        reasons.append(
            f"Amount anomaly: ${event['amount']:.2f} is {z:.1f} standard deviations above this "
            f"account's typical ${account['avg_transaction_amount']:.2f} transaction.")

    # Rule 3: new/foreign country
    if event["country"] != account["home_country"]:
        score += 25
        reasons.append(
            f"Transaction country ({event['country']}) differs from the account's home country "
            f"({account['home_country']}).")

    # Rule 4: unrecognized device
    if event["device_id"] != account["primary_device_id"]:
        score += 20
        reasons.append(
            f"Transaction made from an unrecognized device (expected the account's usual device, "
            f"saw a different one).")

    score = min(score, 100)

```

3.1 The rule set

Rule	Condition	Points	Detects
Velocity	≥ 4 transactions in 60s	+35	bursts of any kind
Amount (severe)	$z \geq 6$ vs the account's own history	+40	a large unusual charge
Amount (moderate)	$z \geq 3$	+20	a somewhat unusual charge
Country	differs from home country	+25	geographic anomaly
Device	differs from primary device	+20	unrecognised hardware

Points accumulate and cap at 100. Bands: **High** at 60 and above, **Medium** at 30, **Low** below that.

3.2 Why the z-score uses the account's own baseline

The amount rule does not ask whether \$2,847 is a large transaction. It asks whether it is large *for this account*:

$$Z = \frac{X - \mu_{\text{account}}}{\max(\sigma_{\text{account}}, 1)}$$

A POPULATION THRESHOLD IS WRONG IN BOTH DIRECTIONS AT ONCE

Set a global limit at \$1,000 and you flag every ordinary transaction of a customer who routinely spends thousands, while missing a \$900 charge on an account whose largest ever purchase was \$60. The first is a false positive factory; the second is the fraud you cared about.

Comparing an account against itself makes the rule adaptive by construction. It is the same instinct behind Project 3's decision to size safety stock on each series' own forecast error rather than on a global variance.

The clamp, and what it protects

`max(std, 1.0)` guards a division by zero. An account with perfectly regular spending has $\sigma = 0$, and without the clamp any deviation would produce an infinite z-score and flag every transaction it ever makes.

THE CLAMP IS A DEFENSIBLE FIX WITH A STATED COST

For a very regular account the clamp makes the rule extremely sensitive - with σ forced to 1, a \$10 deviation from a \$50 norm reads as $z = 10$. That is arguably correct for an account that never varies, and arguably a source of false positives.

What matters is that it is a choice with consequences rather than an accident. The alternative - a floor proportional to the mean, such as $\max(\sigma, 0.1\mu)$ - would be more defensible and is not what was implemented.

3.3 Additive scoring, and its two properties

Rules contribute independent points rather than voting or multiplying. This has one clear virtue and one clear defect, both of which show up in the results.

The virtue: corroboration

Independent weak signals combine into a strong one. Country alone is 25 - not enough to flag. Country plus device is 45. Country plus device plus a severe amount anomaly is 85, comfortably High. `stolen_card_spree` is caught at 100% precisely because three unrelated signals agree.

The defect: no interactions

Additive scoring cannot express "a foreign country is only suspicious on an unrecognised device", or "velocity matters more when amounts are tiny". Every rule contributes the same points regardless of what else is true.

THE DEFECT IS EXACTLY WHAT THE CARD-TESTING CASE NEEDS

Tiny amounts are not suspicious. Home country is not suspicious. The usual device is not suspicious. *Many tiny transactions in seconds* is extremely suspicious - and that is an interaction between velocity and amount which an additive model has no way to represent.

This is the standard argument for moving from hand-written rules to a learned model: a gradient-boosted tree discovers interactions automatically. The counter-argument is that rules are explainable for free, which Chapter 10 shows matters here for legal as well as practical reasons. **Project 12** builds the learned version.

CHAPTER 4

The Sliding Window

Counting events in the last sixty seconds, in constant memory, at streaming speed - and why a sorted set is the right structure.

4.1 The question a velocity rule asks

"How many transactions has this account made in the last sixty seconds?" sounds trivial. At streaming rates, for millions of accounts, with the answer needed in single-digit milliseconds, it is the central engineering problem of the consumer.

Two naive answers fail immediately. **Querying the warehouse** means a database round-trip per event, which is orders of magnitude slower than the 6.2 ms average this pipeline achieves. **Keeping a list per account in memory** works until you have a million accounts and the process restarts, taking all the state with it.

4.2 The sorted-set pattern

The sliding window, in four Redis commands

pipeline/consumer.py

```
pipe.zadd(key, {f"{ts}:{account_id}": ts})
pipe.zremrangebyscore(key, 0, ts - VELOCITY_WINDOW_SECONDS)
pipe.zcard(key)
pipe.expire(key, VELOCITY_WINDOW_SECONDS * 2)
_, _, count, _ = pipe.execute()
return int(count)
```

```
def main():
```

A Redis sorted set holds members ordered by a numeric score. Store the timestamp as the score and the window becomes a range query.

4.1 The four commands and their complexity

Command	What it does	Cost
ZADD	add this event at score = its timestamp	$O(\log n)$
ZREMRANGEBYSCORE	evict everything older than 60s	$O(\log n + m)$
ZCARD	count what remains - the window size	$O(1)$
EXPIRE	let the whole key die if the account goes quiet	$O(1)$

EVICTON ON WRITE IS WHAT BOUNDS THE MEMORY

The set never grows beyond the events in the window, because every write removes the expired ones. There is no cleanup job, no sweeper thread, and no possibility of a forgotten account accumulating state.

The `EXPIRE` is the second line of defence: an account that stops transacting entirely has no writes to trigger eviction, so its key would linger forever holding a handful of stale entries. With a TTL of twice the window, Redis reclaims it automatically. **Across millions of mostly-idle accounts that is the difference between bounded and unbounded memory.**

4.3 Why a pipeline, and why the order matters

All four commands are issued in a Redis **pipeline** - sent in one network round-trip rather than four. At a 6.2 ms budget, three saved round-trips is most of the budget.

ZADD MUST COME BEFORE ZREMRANGEBYSCORE

Add first, then evict, then count - and the count therefore *includes the current event*. That is why the scoring rule reads `recent_count >= 4` and the docstring says 'including this one'.

Reverse the order and every count is one lower, so the rule fires a transaction later than intended - and since Chapter 7 shows the detector is already blind to the first three events of a burst, that off-by-one would make it four. An ordering detail in a Redis pipeline is directly a recall number.

4.4 This is processing-time windowing, and that is a limitation

The window is keyed on the timestamp used at scoring time. If events arrive out of order - a mobile client buffering offline, a network partition healing - the window contains whatever *arrived* in the last sixty seconds, not what *happened* in the last sixty seconds.

THE DISTINCTION THAT PROJECT 14 IS BUILT AROUND

Event time is when the transaction occurred. **Processing time** is when the consumer saw it. They differ by network delay, buffering, retries and clock skew.

A fraudster who could delay their own events could, in principle, spread a burst across two processing windows and keep every count below 4. This pipeline has no watermark, no allowed lateness and no reordering - and at the millisecond latencies measured here the gap is small enough not to matter for the demonstration.

Project 14 takes this seriously and finds that a single fast device clock dragged a max-based watermark 200 seconds ahead and discarded 39,103 of 40,877 messages. The problem is real; it is simply out of scope here.

CHAPTER 5

A Guided Tour of the Codebase

Every module that matters, in the order the pipeline runs them, with its purpose and public surface. This chapter is generated from the repository, so it cannot describe a function that no longer exists.

5.1 How to read this tour

Read the producer first to see what enters the stream, then the consumer to see what happens to it, then the scoring rules on their own. The evaluation script is deliberately last - it runs after everything and cannot influence any of it.

5.2 `scripts/generate_reference_data.py`

Why it exists. Builds the account master: each account's home country, primary device, and the historical spend mean and standard deviation the amount rule compares against.

What the module says about itself. Generates the ACCOUNT REFERENCE data the real-time fraud-scoring rules engine compares every live transaction against: each account's typical spend amount, home country, primary device, and usual merchant categories. Without this "known normal" baseline, a streaming processor can't tell a normal \$40 grocery purchase from a genuinely anomalous one - this is exactly the kind of slowly-changing reference data a real-time system joins against on every event.

Opening of the module

scripts/generate_reference_data.py

```

"""
generate_reference_data.py
-----
Generates the ACCOUNT REFERENCE data the real-time fraud-scoring rules
engine compares every live transaction against: each account's typical
spend amount, home country, primary device, and usual merchant
categories. Without this "known normal" baseline, a streaming processor
can't tell a normal $40 grocery purchase from a genuinely anomalous one -
this is exactly the kind of slowly-changing reference data a real-time
system joins against on every event.

This is a ONE-TIME batch step (reference data doesn't need to be
streamed) - scripts/producer.py and pipeline/consumer.py both read this
file directly.

No real customer or account data is used anywhere in this project.

Output: data/accounts.csv
"""

import numpy as np
import pandas as pd
from pathlib import Path
from datetime import datetime, timedelta

SEED = 7
rng = np.random.default_rng(SEED)

DATA_DIR = Path(__file__).resolve().parent.parent / "data"
DATA_DIR.mkdir(exist_ok=True)

N_ACCOUNTS = 2000

COUNTRIES = ["USA", "Canada", "UK", "Germany", "India", "Australia", "Brazil", "Japan"]
COUNTRY_WEIGHTS = [0.35, 0.10, 0.12, 0.09, 0.12, 0.07, 0.08, 0.07]

CATEGORIES = ["Grocery", "Restaurants", "Fuel", "Electronics", "Travel", "Entertainment",
              "Utilities", "Apparel", "Pharmacy", "Online Retail"]

RISK_TIERS = pd.DataFrame([("Standard", 0.80), ("New Account", 0.12), ("Premium", 0.08)],
                           columns=["tier", "weight"])

account_ids = [f"ACC{300000 + i}" for i in range(N_ACCOUNTS)]
home_countries = rng.choice(COUNTRIES, size=N_ACCOUNTS, p=COUNTRY_WEIGHTS)

# Average transaction size varies a lot by account (this heterogeneity is
# exactly why a flat "flag anything over $500" rule would be naive - a
# premium account's normal $800 purchase and a student account's normal
# $15 purchase need different baselines).
avg_amounts = rng.lognormal(mean=3.9, sigma=0.7, size=N_ACCOUNTS).clip(8, 900)
std_amounts = avg_amounts * rng.uniform(0.25, 0.45, size=N_ACCOUNTS) # typical spend variability

primary_devices = [f"DEV{rng.integers(100000, 999999)}" for _ in range(N_ACCOUNTS)]

# Each account has 2-4 categories it normally shops in
typical_categories = [
    ", ".join(rng.choice(CATEGORIES, size=int(rng.integers(2, 5))), replace=False))

```

```

    for _ in range(N_ACCOUNTS)
]

risk_tiers = rng.choice(RISK_TIERS.tier, size=N_ACCOUNTS, p=RISK_TIERS.weight)
opened_dates = [(datetime(2026, 7, 30) - timedelta(days=int(d))).date().isoformat()
                 for d in rng.integers(15, 2200, size=N_ACCOUNTS)]

accounts = pd.DataFrame({
    "account_id": account_ids,
    "home_country": home_countries,
    "avg_transaction_amount": avg_amounts.round(2),
    "std_transaction_amount": std_amounts.round(2),
    "primary_device_id": primary_devices,
    "typical_categories": typical_categories,
    "risk_tier": risk_tiers,
})

```

5.3 scripts/producer.py

Why it exists. The live feed. Emits normal traffic plus the three fraud scenarios, XADding each event to the Redis stream. Every event carries the scenario label that the scoring code never sees.

What the module says about itself. Simulates a live transaction feed and XADds each event to the Redis Stream `transactions:raw` in real time (not a batch dump) - this is the "event source" half of the pipeline, standing in for what a real payments switch or point-of-sale gateway would publish.

Public functions in scripts/producer.py

Function	What it does
load_accounts()	—
make_txn_id()	—
normal_event(account, rng)	—
card_testing_events(account, rng)	—
stolen_card_spree_events(account, rng)	—
account_takeover_events(account, rng)	—
main()	—

Opening of the module

scripts/producer.py

```

"""
producer.py
-----

Simulates a live transaction feed and XADDs each event to the Redis
Stream `transactions:raw` in real time (not a batch dump) - this is the
"event source" half of the pipeline, standing in for what a real
payments switch or point-of-sale gateway would publish.

Emits mostly normal transactions at a steady baseline rate, with a
handful of scheduled FRAUD SCENARIOS injected at random points during
the run:

- card_testing:      a burst of 5-8 tiny transactions in <1s gaps
                      (an attacker probing whether a stolen card
                      number still works, before attempting a big
                      purchase)
- stolen_card_sprees: 3-5 large transactions from a foreign country
                      on an unrecognized device, in quick succession
- account_takeover:   a device change followed by a burst of
                      escalating-amount transactions from the
                      account's own home country (the sneakier
                      pattern - it doesn't trip an obvious
                      new-country flag, only velocity + new-device
                      + amount anomaly)

Each event carries a `scenario_label` field for LATER EVALUATION ONLY
(scripts/evaluate_detection.py) - pipeline/consumer.py's scoring logic
never reads this field, so the rules are judged fairly, the same way
they'd have to work against a real, unlabeled live feed.

Usage:
    python scripts/producer.py --duration 180 --rate 2.5
"""

import argparse
import json
import time
import uuid
from pathlib import Path

import numpy as np
import pandas as pd

import sys
ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT))
from pipeline.redis_client import get_client, STREAM_RAW # noqa: E402

DATA_DIR = ROOT / "data"
MERCHANTS = [f"MERCH{i:04d}" for i in range(500)]
FOREIGN_COUNTRIES = ["Nigeria", "Russia", "Ukraine", "Vietnam", "Romania", "Indonesia"]

def load_accounts():
    df = pd.read_csv(DATA_DIR / "accounts.csv")
    df["typical_categories"] = df["typical_categories"].str.split(",")
    return df

```

```
def make_txn_id():
    return f"TXN{uuid.uuid4().hex[:12]}"

def normal_event(account, rng):
    amount = max(1.0, rng.lognormal(np.log(account.avg_transaction_amount), 0.35))
    category = rng.choice(account.typical_categories)
    return {
        "transaction_id": make_txn_id(),
        "account_id": account.account_id,
        "amount": round(float(amount), 2),
        "country": account.home_country,
        "device_id": account.primary_device_id,
```

5.4 pipeline/redis_client.py

Why it exists. Connection handling, kept in one place so the transport is swappable.

What the module says about itself. Shared connection + naming constants for every part of this pipeline that talks to Redis. Redis was compiled from the official source (download.redis.io) and run directly - no Docker, no Windows service installer - because the RAM-friendlier Windows path (Memurai) hit an installer bug on this machine (see docs/architecture.md). WSL already had gcc/make available, so a from-source build was the most reliable path forward without needing sudo/apt.

Public functions in pipeline/redis_client.py

Function	What it does
get_client()	—

Opening of the module

pipeline/redis_client.py

```

"""
redis_client.py
-----
Shared connection + naming constants for every part of this pipeline
that talks to Redis. Redis was compiled from the official source
(download.redis.io) and run directly - no Docker, no Windows service
installer - because the RAM-friendlier Windows path (Memurai) hit an
installer bug on this machine (see docs/architecture.md). WSL already
had gcc/make available, so a from-source build was the most reliable
path forward without needing sudo/apt.
"""

import os
import redis

REDIS_HOST = os.environ.get("REDIS_HOST", "localhost")
REDIS_PORT = int(os.environ.get("REDIS_PORT", 6379))

STREAM_RAW = "transactions:raw"
CONSUMER_GROUP = "fraud-scorers"
CONSUMER_NAME = "scorer-1"

VELOCITY_KEY_PREFIX = "velocity:"          # sorted-set sliding window per account
VELOCITY_WINDOW_SECONDS = 60

def get_client() -> redis.Redis:
    return redis.Redis(host=REDIS_HOST, port=REDIS_PORT, decode_responses=True)

```

5.5 pipeline/consumer.py

Why it exists. The heart of the system: XREADGROUP, the four-command sliding window, scoring, latency measurement, the warehouse write, and XACK.

What the module says about itself. The real-time stream processor: reads transaction events off the Redis Stream via a CONSUMER GROUP (XREADGROUP / XACK) - not a naive poll-and- delete - so message processing is at-least-once and acknowledged individually, the same durability pattern a real production consumer would use (if this process crashes mid-batch, un-ACKed messages are still claimable by a replacement consumer instead of being lost).

Public functions in pipeline/consumer.py

Function	What it does
ensure_consumer_group(client)	—
update_velocity(client, account_id, ts)	—
main()	—

Opening of the module

pipeline/consumer.py

```

"""
consumer.py
-----

The real-time stream processor: reads transaction events off the Redis
Stream via a CONSUMER GROUP (XREADGROUP / XACK) - not a naive poll-and-
delete - so message processing is at-least-once and acknowledged
individually, the same durability pattern a real production consumer
would use (if this process crashes mid-batch, un-ACKed messages are
still claimable by a replacement consumer instead of being lost).

For each event:
    1. Look up the account's historical profile (loaded once, in memory).
    2. Update a Redis SORTED SET sliding-window velocity counter for that
       account (ZADD + ZREMRANGEBYSCORE + ZCARD) - a standard, idiomatic
       Redis pattern for real-time rate/velocity tracking.
    3. Score the transaction (pipeline/scoring_rules.py).
    4. Record end-to-end latency (now vs. the event's own timestamp) -
       the actual, measured proof this is real-time, not batch.
    5. Write the scored result to DuckDB and XACK the message.

Usage:
    python pipeline/consumer.py --duration 200
(run concurrently with scripts/producer.py, or after it - either order
is safe since the consumer group starts reading from the beginning)
"""

import argparse
import sys
import time
from pathlib import Path

import duckdb
import pandas as pd
import redis

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT))
from pipeline.redis_client import (get_client, STREAM_RAW, CONSUMER_GROUP, CONSUMER_NAME, # noqa: E402
                                  VELOCITY_KEY_PREFIX, VELOCITY_WINDOW_SECONDS)
from pipeline.scoring_rules import score_transaction # noqa: E402

DATA_DIR = ROOT / "data"
DB_PATH = ROOT / "database" / "streaming.duckdb"

SCHEMA = """
CREATE TABLE IF NOT EXISTS transaction_scores (
    transaction_id TEXT PRIMARY KEY,
    account_id TEXT,
    amount DOUBLE,
    country TEXT,
    device_id TEXT,
    category TEXT,
    merchant_id TEXT,
    event_ts DOUBLE,
    processed_ts DOUBLE,
    latency_ms DOUBLE,
    recent_count INTEGER,

```



```

risk_score INTEGER,
risk_level TEXT,
reasons TEXT,
scenario_label TEXT
);
"""

def ensure_consumer_group(client: redis.Redis):
    try:
        client.xgroup_create(STREAM_RAW, CONSUMER_GROUP, id="0", mkstream=True)
        print(f"Created consumer group '{CONSUMER_GROUP}' on stream '{STREAM_RAW}'")
    except redis.exceptions.ResponseError as e:
        if "BUSYGROUP" in str(e):
            print(f"Consumer group '{CONSUMER_GROUP}' already exists - resuming.")

```

5.6 pipeline/scoring_rules.py

Why it exists. The four rules, isolated from all Redis plumbing so they can be unit-tested - and so the fields they receive are auditable.

What the module says about itself. The rules-based fraud scoring logic, kept separate from the Redis plumbing in consumer.py so it can be unit-tested and read on its own.

Public functions in pipeline/scoring_rules.py

Function	What it does
score_transaction(event, account, recent_count)	event: dict with amount, country, device_id, category, account_id

Opening of the module

pipeline/scoring_rules.py

```

"""
scoring_rules.py
-----

The rules-based fraud scoring logic, kept separate from the Redis
plumbing in consumer.py so it can be unit-tested and read on its own.

4 rules, each contributing points to a 0-100 risk score, and each
producing a plain-English reason string - which is what makes the
downstream AI explanation feature possible without an LLM having to
invent anything: the "why" is already computed here, in code, and the
LLM's job (pipeline/explain.py) is only to turn a list of factual
reasons into a readable narrative, not to guess at causes itself.

IMPORTANT: this function only ever sees the fields a real fraud system
would actually have at transaction time (amount, country, device,
category, timestamp, and the account's own historical profile) - it
never sees `scenario_label`, which exists purely for later evaluation
(scripts/evaluate_detection.py). That separation is what makes the
evaluation honest.
"""

RISK_THRESHOLDS = {"High": 60, "Medium": 30}

def score_transaction(event: dict, account: dict, recent_count: int) -> dict:
    """
    event: dict with amount, country, device_id, category, account_id
    account: dict with avg_transaction_amount, std_transaction_amount,
                home_country, primary_device_id
    recent_count: number of transactions from this account in the last
                  60 seconds (INCLUDING this one), from the Redis
                  sliding-window sorted set - see consumer.py
    """
    score = 0
    reasons = []

    # Rule 1: velocity
    if recent_count >= 4:
        score += 35
        reasons.append(
            f"High velocity: {recent_count} transactions from this account in the last 60 seconds "
            f"(a normal account rarely exceeds 2-3)."
        )

    # Rule 2: amount anomaly (z-score vs. this account's own historical baseline)
    std = max(account["std_transaction_amount"], 1.0)
    z = (event["amount"] - account["avg_transaction_amount"]) / std
    if z > 6:
        score += 40
        reasons.append(
            f"Severe amount anomaly: ${event['amount']:.2f} is {z:.1f} standard deviations above "
            f"this account's typical ${account['avg_transaction_amount']:.2f} transaction."
        )
    elif z > 3:
        score += 20
        reasons.append(
            f"Amount anomaly: ${event['amount']:.2f} is {z:.1f} standard deviations above this "

```

```

        f"account's typical ${account['avg_transaction_amount']:.2f} transaction."
    )

    # Rule 3: new/foreign country
    if event["country"] != account["home_country"]:
        score += 25
        reasons.append(
            f"Transaction country ({event['country']}) differs from the account's home country "
            f"({account['home_country']})."
        )

    # Rule 4: unrecognized device
    if event["device_id"] != account["primary_device_id"]:
        score += 20
        reasons.append(

```

5.7 pipeline/explain.py

Why it exists. Turns the reason strings the rules already computed into an analyst-readable narrative with a local model. The model never infers a cause.

What the module says about itself. Turns a flagged transaction's rule-based reasons (already fully computed, factual strings from `scoring_rules.py`) into a short, readable fraud-analyst-style narrative using a local Ollama model - the same `qwen2.5:0.5b` used in Project 5, for the same reason (this machine's ~6GB RAM budget, and so this project costs nothing to run for anyone, ever).

Public functions in `pipeline/explain.py`

Function	What it does
<code>explain_alert(transaction)</code>	transaction: a dict/row with amount, category, country, account_id,

Opening of the module

pipeline/explain.py

```

"""
explain.py
-----
Turns a flagged transaction's rule-based reasons (already fully
computed, factual strings from scoring_rules.py) into a short, readable
fraud-analyst-style narrative using a local Ollama model - the same
qwen2.5:0.5b used in Project 5, for the same reason (this machine's ~6GB
RAM budget, and so this project costs nothing to run for anyone, ever).

Deliberately NOT a RAG/retrieval setup Like Project 5's copilot - there's
no ambiguity to retrieve context for here. The rules already produced
the exact, factual reasons; the LLM's only job is turning a bullet list
into a paragraph a human reads faster than a bullet list, and suggesting
one concrete next action. It is given the reasons as fact and instructed
not to invent anything beyond them.
"""

import ollama

OLLAMA_MODEL = "qwen2.5:0.5b"

PROMPT_TEMPLATE = """A fraud detection system flagged a transaction with risk level {risk_level} \
(score {risk_score}/100). The SPECIFIC reasons it was flagged are listed below - these are the \
only facts you know; do not invent any other reason.

Transaction: ${amount:.2f} at a {category} merchant in {country}, account {account_id}.

Reasons flagged:
{reasons_bullets}

Write a 2-3 sentence summary a fraud analyst could read in 5 seconds, explaining why this was \
flagged and suggesting one concrete next action (e.g. "hold for manual review", "contact the \
cardholder to verify", "likely safe to auto-approve if this pattern repeats legitimately"). \
Do not repeat the raw numbers unnecessarily - write it as a person would explain it verbally."""

def explain_alert(transaction: dict) -> str:
    """transaction: a dict/row with amount, category, country, account_id,
    risk_level, risk_score, reasons (list of strings)."""
    reasons_bullets = "\n".join(f"- {r}" for r in transaction["reasons"]) or "- (no specific reasons -
    low risk)"

    prompt = PROMPT_TEMPLATE.format(
        risk_level=transaction["risk_level"],
        risk_score=transaction["risk_score"],
        amount=transaction["amount"],
        category=transaction["category"],
        country=transaction["country"],
        account_id=transaction["account_id"],
        reasons_bullets=reasons_bullets,
    )

    response = ollama.chat(
        model=OLLAMA_MODEL,
        messages=[{"role": "user", "content": prompt}],
        options={"temperature": 0.2, "num_predict": 220},
    )

```

```

return response["message"]["content"].strip()

if __name__ == "__main__":
    demo = {
        "account_id": "ACC300123", "amount": 1450.00, "category": "Electronics",
        "country": "Russia", "risk_level": "High", "risk_score": 85,
        "reasons": [
            "Severe amount anomaly: $1450.00 is 8.2 standard deviations above this account's "
            "typical $120.00 transaction.",
            "Transaction country (Russia) differs from the account's home country (USA).",
            "Transaction made from an unrecognized device (expected the account's usual device, "
            "saw a different one).",
        ],
    }
    print(explain_alert(demo))

```

5.8 scripts/evaluate_detection.py

Why it exists. Runs after the fact, joins the scored warehouse against the injected labels, and computes the confusion matrix, per-scenario catch rates and latency percentiles.

What the module says about itself. Measures how well the rules engine actually did, using the `scenario\_label` ground-truth field the producer attached to each event purely for this evaluation - and which pipeline/scoring_rules.py never sees or uses while scoring. This is the same "inject a known incident, then measure detection against it" pattern used in every prior project in this portfolio, applied here to a real-time detection rate instead of a batch data-quality check.

Opening of the module

scripts/evaluate_detection.py

```

"""
evaluate_detection.py
-----
Measures how well the rules engine actually did, using the
`scenario_label` ground-truth field the producer attached to each event
purely for this evaluation - and which pipeline/scoring_rules.py never
sees or uses while scoring. This is the same "inject a known incident,
then measure detection against it" pattern used in every prior project
in this portfolio, applied here to a real-time detection rate instead
of a batch data-quality check.

Ground truth: any event with scenario_label != "normal" is fraud.
Prediction: any event with risk_level in (Medium, High) is flagged.

Output: reports/detection_evaluation.json + .md
"""

import json
from pathlib import Path

import duckdb
import pandas as pd

ROOT = Path(__file__).resolve().parent.parent
DB_PATH = ROOT / "database" / "streaming.duckdb"
REPORTS_DIR = ROOT / "reports"
REPORTS_DIR.mkdir(exist_ok=True)

conn = duckdb.connect(str(DB_PATH), read_only=True)
df = conn.execute("SELECT * FROM transaction_scores").fetchdf()
conn.close()

df["is_fraud"] = df["scenario_label"] != "normal"
df["is_flagged"] = df["risk_level"].isin(["Medium", "High"])

tp = int(((df.is_fraud) & (df.is_flagged)).sum())
fp = int(((~df.is_fraud) & (df.is_flagged)).sum())
fn = int(((df.is_fraud) & (~df.is_flagged)).sum())
tn = int(((~df.is_fraud) & (~df.is_flagged)).sum())

precision = tp / (tp + fp) if (tp + fp) else 0
recall = tp / (tp + fn) if (tp + fn) else 0
f1 = 2 * precision * recall / (precision + recall) if (precision + recall) else 0

by_scenario = df[df.is_fraud].groupby("scenario_label").agg(
    total=("transaction_id", "count"),
    caught=("is_flagged", "sum"),
).reset_index()
by_scenario["catch_rate"] = (by_scenario.caught / by_scenario.total * 100).round(1)

latency_stats = {
    "avg_latency_ms": round(df.latency_ms.mean(), 1),
    "p95_latency_ms": round(df.latency_ms.quantile(0.95), 1),
    "max_latency_ms": round(df.latency_ms.max(), 1),
}

summary = {

```

```

    "total_events": len(df),
    "total_fraud_events": int(df.is_fraud.sum()),
    "total_flagged": int(df.is_flagged.sum()),
    "confusion_matrix": {"true_positive": tp, "false_positive": fp, "false_negative": fn,
    "true_negative": tn},
    "precision": round(precision, 3),
    "recall": round(recall, 3),
    "f1_score": round(f1, 3),
    "by_scenario": by_scenario.to_dict(orient="records"),
    "latency": latency_stats,
}

with open(REPORTS_DIR / "detection_evaluation.json", "w", encoding="utf-8") as f:
    json.dump(summary, f, indent=2)

with open(REPORTS_DIR / "detection_evaluation.md", "w", encoding="utf-8") as f:

```

5.9 dashboard/streamlit_app.py

Why it exists. The operations console: flagged queue, score distribution, latency, and the reasons behind each flag.

What the module says about itself. Live-monitoring dashboard for the real-time fraud-scoring pipeline, plus an evaluation tab and an on-demand AI explanation for any flagged transaction. Reads directly from database/streaming.duckdb - the table pipeline/consumer.py writes to as it processes the Redis Stream live.

Public functions in dashboard/streamlit_app.py

Function	What it does
load_data()	—

Opening of the module

dashboard/streamlit_app.py

```

"""
streamlit_app.py
-----
Live-monitoring dashboard for the real-time fraud-scoring pipeline, plus
an evaluation tab and an on-demand AI explanation for any flagged
transaction. Reads directly from database/streaming.duckdb - the table
pipeline/consumer.py writes to as it processes the Redis Stream Live.

Run with:
    streamlit run dashboard/streamlit_app.py

For the "live" feel, enable "Live Mode" in the sidebar while
scripts/producer.py + pipeline/consumer.py are running concurrently in
other terminals - the dashboard will poll and refresh automatically.
"""

import json
import sys
import time
from pathlib import Path

import duckdb
import pandas as pd
import plotly.express as px
import streamlit as st

ROOT = Path(__file__).resolve().parent.parent
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from pipeline.explain import explain_alert # noqa: E402

DB_PATH = ROOT / "database" / "streaming.duckdb"
REPORTS_DIR = ROOT / "reports"

st.set_page_config(page_title="Real-Time Fraud Monitor", layout="wide", page_icon="🔥")

def load_data():
    if not DB_PATH.exists():
        return pd.DataFrame()
    conn = duckdb.connect(str(DB_PATH), read_only=True)
    df = conn.execute("SELECT * FROM transaction_scores ORDER BY event_ts DESC").fetchdf()
    conn.close()
    if len(df):
        df["reasons_list"] = df["reasons"].apply(lambda s: [r.strip() for r in s.split("|")] if s else [])
    return df

st.sidebar.title("Controls")
live_mode = st.sidebar.checkbox("🔴 Live Mode (auto-refresh every 3s)", value=False)
st.sidebar.caption("Run `python scripts/producer.py` and `python pipeline/consumer.py` in separate "
                  "terminals, then enable Live Mode here to watch the feed update in real time.")

df = load_data()

```



```

st.title("⚡ Real-Time Transaction Fraud Monitor")
st.caption("Redis Streams (consumer group, XREADGROUP/XACK) → rules-based scoring → DuckDB. "
          "Every number below is read live from what the streaming consumer has actually processed.")

tab_monitor, tab_eval = st.tabs(["📊 Live Monitor", "🎯 Detection Evaluation"])

# =====
# TAB 1: Live Monitor
# =====
with tab_monitor:
    if df.empty:
        st.info("No events processed yet. Start the producer and consumer, then refresh.")
    else:
        k1, k2, k3, k4, k5 = st.columns(5)
        k1.metric("Events Processed", f"{len(df):,}")
        flagged = df[df.risk_level.isin(["Medium", "High"])]

```

CHAPTER 6

Evaluating a Detector Honestly

Precision, recall and F1 derived; why the labels must be invisible to the scorer; and what these numbers do not transfer to.

6.1 The four outcomes

6.1 The confusion matrix from the real run

	Actually fraud	Actually normal
Flagged	true positive - 35	false positive - 1
Not flagged	false negative - 9	true negative - 370

$$\text{precision} = \frac{TP}{TP + FP} = \frac{35}{36} = 0.972$$

$$\text{recall} = \frac{TP}{TP + FN} = \frac{35}{44} = 0.795$$

$$F_1 = \frac{2 \cdot P \cdot R}{P + R} = \frac{2(0.972)(0.795)}{0.972 + 0.795} = 0.875$$

Precision answers "when we flag, are we right?" - it is about the analyst's time and the customer's inconvenience. Recall answers "of the fraud that happened, how much did we catch?" - it is about money lost. They trade against each other, and the F1 is their harmonic mean.

THE HARMONIC MEAN PUNISHES IMBALANCE ON PURPOSE

A detector with precision 1.0 and recall 0.1 has an arithmetic mean of 0.55 and an F1 of 0.18. The harmonic mean is dragged toward the smaller value, which is the right behaviour: a detector that is always right about the 10% it catches is not a good detector.

F1 also weights the two equally, which is a decision rather than a law. Given the asymmetric costs in Chapter 1, a fraud team would often prefer F_β with $\beta > 1$ to weight recall more heavily.

6.2 What makes this evaluation trustworthy

1. **The ground truth was injected before the detector existed.** The producer plants three named scenarios; the scorer is graded against them. Nothing was labelled after the fact by looking at what the detector flagged.

2. **The scorer cannot see the labels.** `score_transaction` receives amount, country, device, category, timestamp and the account profile. `scenario_label` is never passed in.
3. **The evaluation is a separate program.** `scripts/evaluate_detection.py` reads the warehouse after the fact and joins on the label. It cannot influence scoring because it runs afterwards.
4. **Per-scenario results are reported.** Which, as Chapter 7 shows, is where the actual finding lives.

LABEL LEAKAGE IS A SINGLE FIELD AWAY

Adding `scenario_label` to the dictionary passed into `score_transaction` would take one character of diff and produce a detector with perfect precision and recall. Nothing in the output would look wrong - the metrics would simply be excellent.

This is why the constraint is written into the module docstring rather than left as an unstated convention. **The strongest protection against leakage is making the boundary explicit enough that crossing it requires a deliberate act.**

6.3 What these numbers do not establish

1. **That they transfer to real fraud.** The base rate here is 10.6%; real card fraud is nearer 0.1%. At that rate the single false positive becomes proportionally far more costly, and precision would collapse even with an unchanged detector.
2. **That the rules generalise.** They were written knowing the three scenarios exist. That is not leakage in the technical sense, but it is a strong prior, and a fourth attack type would find its own gaps.
3. **That 415 events is enough to estimate anything.** One false positive gives a 95% Wilson interval on precision of roughly [86%, 99.5%] - a wide band. The single-FP figure is a point estimate with very little behind it.
4. **That the latency holds under load.** One producer, one consumer, no contention, no network.
5. **That the detector is at-least-once safe.** It acknowledges after writing, so a duplicate delivery would double-score and double-count the velocity window.

THE ONE CLAIM THAT DOES TRANSFER

The structural findings are independent of the data volume. **Any velocity rule with threshold k is blind to the first $k - 1$ events of any burst**, and any additive rule set is unable to express interactions. Those are properties of the design, and they would hold identically on a billion real transactions.

CHAPTER 7

The Recall Ceiling

Why the detector misses exactly nine events, why that number was predictable in advance, and why the single false positive is indistinguishable from fourteen true ones.

7.1 Where the misses are

7.1 Catch rate by scenario

Scenario	Events	Caught	Catch rate
account_takeover	6	6	100%
stolen_card_spree	15	15	100%
card_testing	23	14	60.9%
Total	44	35	79.5%

All nine false negatives are card testing. Two scenarios are caught perfectly and one is not, so the aggregate recall of 79.5% is not a property of the detector in general - it is one scenario's failure, averaged in.

THIS IS THE AGGREGATION TRAP OF PROJECT 5, IN A DIFFERENT COSTUME

A single headline recall number pools three scenarios with catch rates of 100%, 100% and 60.9%. Reported alone it suggests a detector that is uniformly a bit leaky, when in fact it is perfect at two attack types and structurally weak at a third.

Always report detection metrics per attack class. The aggregate is the number an adversary least cares about, because they will simply use the attack you are worst at.

7.2 Why card testing is nearly invisible

Look at what a card-testing transaction actually is: \$1-\$5, in the account's home country, on the account's usual device. Now check it against each rule.

7.2 Three of four rules are structurally unavailable

Rule	On a card-testing event	Fires?
Amount $z \geq 3$	amount is far below average; z is negative	never
Country differs	attacker uses the home country	never
Device differs	attacker uses the usual device	never
Velocity ≥ 4	the burst is 5-8 events in seconds	only rule available

The warehouse confirms it exactly: every one of the 23 card-testing events scored either **0** or **35**. Never 20, never 55, never anything else - because only one rule can ever contribute.

7.3 The warm-up, derived

The velocity rule needs $k = 4$ events in the window. Within a burst the count goes 1, 2, 3, 4, ... so the rule cannot fire on the first three events. For a burst of b :

$$\text{missed} = k - 1 = 3, \quad \text{recall}_{\text{burst}} = 1 - \frac{k-1}{b}$$

The three bursts in this run had 7, 8 and 8 events. The warehouse shows their score sequences:

Every burst, straight from the warehouse

```
ACC300960: n=7  scores = [0, 0, 0, 35, 35, 35, 35]      missed 3
ACC301100: n=8  scores = [0, 0, 0, 35, 35, 35, 35, 35]  missed 3
ACC301922: n=8  scores = [0, 0, 0, 35, 35, 35, 35, 35]  missed 3
```

60.9% IS PREDICTABLE FROM THE RULE ALONE, WITH NO DATA

Three bursts, three missed each, nine total. 14 of 23 caught is **60.9%** - the reported figure, derived from nothing but $k = 4$ and the burst sizes.

And since the other two scenarios are at 100%, overall recall is $35/44 = 79.5\%$ - which means **the detector caught every single fraud event it was capable of catching**. 79.5% is not a tuning shortfall. It is the ceiling, and the detector is sitting on it.

Why lowering k is not the answer

Setting $k = 2$ would recover two events per burst. It would also fire on any account making two transactions in a minute - buying coffee and a newspaper - and the false positive rate would swamp the gain. The velocity rule's threshold trades warm-up recall against false positives, and there is no setting that removes the warm-up entirely: **any velocity rule is blind to the beginning of the burst it is designed to detect**.

WHAT WOULD ACTUALLY FIX IT

A signal that fires on the *first* tiny transaction. Two candidates: an **amount-below-normal** rule, since a \$3 charge on an account averaging \$84 is itself unusual and the current rule tests only the upper tail; or a **merchant-category-plus-amount** interaction, since micro-charges at online retail are the signature of a card test.

Both are new information rather than new tuning, which is the general lesson: **when a detector is at its structural ceiling, the next improvement is a feature, not a threshold.**

7.4 What an attacker does with this knowledge

The warm-up formula $1 - (k - 1)/b$ has a variable the defender does not control: b , the burst size. The attacker chooses it.

7.4 Recall as a function of a variable the attacker controls

Attacker's burst size	Missed per burst	Caught	Detector recall
8 (as observed)	3	5	62.5%
6	3	3	50.0%
4	3	1	25.0%
3	3	0	0.0%

AN ATTACKER SENDING BURSTS OF THREE IS COMPLETELY INVISIBLE

Not 'hard to catch' - invisible. With $k = 4$, a burst of three never reaches the threshold, so every event scores zero and the detector reports a clean run.

Testing eight card numbers as one burst of eight gives the defender five flags. Testing the same eight as three separate bursts of three, seconds apart, gives the defender **nothing** - and costs the attacker almost nothing, because the window is only sixty seconds wide and they were going to wait anyway.

This is why a threshold-based rule is a weak defence against a motivated adversary: the threshold is public in effect, because it can be discovered by probing, and once known it defines a safe operating region. Lowering k moves the region but does not remove it, and each step down costs false positives on legitimate bursts.

THE GENERAL SHAPE OF THE PROBLEM

Any detector with a fixed threshold on a quantity the attacker controls has a region in which the attacker is safe by construction. The defence is not a better threshold but a signal the attacker cannot cheaply manipulate - here, the amount pattern, which they cannot change without abandoning the probe.

Project 15 shows the same dynamic measured end to end: a fraud detector scoring 95.5% recall collapses to 18.2% under the cheapest adversarial adaptation available. A detector evaluated only against a fixed scenario set has not been evaluated against an adversary at all.

7.5 The false positive nobody can distinguish

Precision is 97.2% because there is exactly one false positive. It is worth looking at.

The single false positive, from the warehouse

```
risk_score: 35  risk_level: Medium
reasons: "High velocity: 8 transactions from this account in the last
        60 seconds (a normal account rarely exceeds 2-3)."
```

IT IS BYTE-IDENTICAL IN FORM TO FOURTEEN TRUE POSITIVES

Same score of 35. Same single reason. Same rule fired. The only difference is the `scenario_label` - which the detector cannot see and which does not exist in production.

The rule set genuinely cannot separate a bursting legitimate account from a card-testing attack. Not 'separates it poorly' - it produces literally the same output for both. Precision of 97.2% is achieved only because normal accounts rarely burst, not because the detector can tell the difference.

What the score bands actually look like

7.3 Precision by score band

Score	True positives	False positives	Precision in band
100	3	0	100%
95	4	0	100%
85	12	0	100%
60	2	0	100%
35	14	1	93.3%
0	0 (9 missed fraud)	370	-

Everything at 60 and above is fraud with no exceptions. All the ambiguity - and all the errors in both directions - lives in the 35-point band, where exactly one rule has fired.

THE SYSTEM IS FIVE POINTS FROM BEING BLIND TO AN ENTIRE ATTACK CLASS

Card testing scores exactly 35. The Medium band starts at 30. If anyone raised the alerting threshold to High - a completely ordinary response to too many alerts - card-testing recall would drop from 60.9% to **zero**, and the metric that would improve is precision, which would go to 100%.

A team watching precision rise and alert volume fall would reasonably conclude they had improved the system. They would have switched off detection for one of the three attacks they face. **A detector whose entire response to an attack class sits five points above a threshold is fragile in a way no summary metric reveals.**

CHAPTER 8

Measuring Latency Honestly

What 'real-time' means when you have to prove it, why the average is the least useful number, and what these measurements do and do not include.

8.1 Measure end to end, not stage by stage

The consumer records latency as the difference between the event's own timestamp and the moment the scored result is written to the warehouse. Everything in between is included: the stream hop, the consumer group read, the four Redis window commands, the scoring, and the database write.

$$\text{latency} = t_{\text{written}} - t_{\text{event}}$$

A PER-STAGE MEASUREMENT WOULD FLATTER THE RESULT

It is easy to report that scoring takes 40 microseconds. It is also meaningless: the user cares when the answer is *available*, and the time between stages - queueing, serialisation, waiting - is usually where the time actually goes.

Measuring from the event's own timestamp also means the number cannot be improved by making the pipeline lazier. It is the honest denominator.

8.2 The measured distribution

8.1 End-to-end latency across the run

Statistic	Value	What it tells you
average	6.2 ms	the typical case, and the least useful number
p95	21.1 ms	what one event in twenty experiences
max	30.4 ms	the worst case observed

THE P95 IS 3.4X THE AVERAGE, AND THAT RATIO IS THE REAL STORY

A latency distribution is never symmetric. It has a floor - work has to happen - and a long right tail from garbage collection, occasional disk flushes, and scheduler preemption.

Reporting only the 6.2 ms average would describe a system nobody experiences. Five percent of transactions take at least three times longer, and it is the tail that breaks an SLA. **Percentiles are the honest summary of a latency distribution; the mean is a summary of a distribution that latency never has.**

Even the max, at 30.4 ms, is comfortably inside any plausible real-time budget - a card authorisation typically allows a few hundred milliseconds end to end. The claim 'this is real-time, not batch' is supported by a measurement rather than by an architecture diagram.

8.3 What these numbers do not include

1. **Network.** Producer, Redis, consumer and warehouse are all on one machine. Real deployments add a network hop at every arrow.
2. **Contention.** One consumer, one producer, a few hundred events. Nothing here is under load.
3. **Cold start.** Connections were already open and the process warm.
4. **Backpressure.** The consumer kept up comfortably, so no queue ever built. The interesting latency behaviour of a streaming system is what happens when it *cannot* keep up - and that was not tested.

THE REGIME THAT WAS MEASURED IS THE EASY ONE

These figures characterise a system with slack. The failure mode of streaming systems is not being slightly slow; it is falling behind, at which point lag grows without bound and latency is limited by the backlog rather than by the code.

A load test - increase the rate until the consumer cannot keep up, and observe where the knee is - is the missing measurement, and it is the one that would tell you the system's actual capacity.

CHAPTER 9

The Explanation Layer

Why the reasons are computed in code and only the phrasing is left to the model - the architecture that makes a tiny LLM safe here.

9.1 The division of labour

Every rule that fires appends a plain-English reason string, built from the actual numbers, in Python. By the time the LLM is involved, the explanation already exists as facts.

Reasons, computed in code before any model sees them

```
High velocity: 8 transactions from this account in the last 60 seconds
(a normal account rarely exceeds 2-3).
Severe amount anomaly: $2,847.19 is 9.4 standard deviations above this
account's typical $84.20 transaction.
Transaction country (RO) differs from the account's home country (IN).
```

THE MODEL IS NEVER ASKED WHY - ONLY HOW TO SAY IT

A model asked "why was this transaction flagged?" must infer a cause, and a 0.5-billion-parameter model asked to infer a cause about money will invent one fluently.

A model handed three factual reasons and asked to write them as a paragraph for an analyst is doing translation, not reasoning. **The correctness lives in the scoring code; the model contributes only fluency.** If it drifts, it drifts in phrasing over facts that are on the record beside it.

This is the same architecture as Project 5's copilot - grounding carries the truth and the model carries the language - arrived at for the same reason: it is what makes a tiny local model responsible for something that matters.

The explanation layer

pipeline/explain.py

```

"""
explain.py
-----
Turns a flagged transaction's rule-based reasons (already fully
computed, factual strings from scoring_rules.py) into a short, readable
fraud-analyst-style narrative using a local Ollama model - the same
qwen2.5:0.5b used in Project 5, for the same reason (this machine's ~6GB
RAM budget, and so this project costs nothing to run for anyone, ever).

Deliberately NOT a RAG/retrieval setup Like Project 5's copilot - there's
no ambiguity to retrieve context for here. The rules already produced
the exact, factual reasons; the LLM's only job is turning a bullet list
into a paragraph a human reads faster than a bullet list, and suggesting
one concrete next action. It is given the reasons as fact and instructed
not to invent anything beyond them.
"""

import ollama

OLLAMA_MODEL = "qwen2.5:0.5b"

PROMPT_TEMPLATE = """A fraud detection system flagged a transaction with risk level {risk_level} \
(score {risk_score}/100). The SPECIFIC reasons it was flagged are listed below - these are the \
only facts you know; do not invent any other reason.

Transaction: ${amount:.2f} at a {category} merchant in {country}, account {account_id}.

Reasons flagged:
{reasons_bullets}

Write a 2-3 sentence summary a fraud analyst could read in 5 seconds, explaining why this was \
flagged and suggesting one concrete next action (e.g. "hold for manual review", "contact the \
cardholder to verify", "likely safe to auto-approve if this pattern repeats legitimately"). \
Do not repeat the raw numbers unnecessarily - write it as a person would explain it verbally."""

def explain_alert(transaction: dict) -> str:
    """transaction: a dict/row with amount, category, country, account_id,
    risk_level, risk_score, reasons (list of strings)."""
    reasons_bullets = "\n".join(f"- {r}" for r in transaction["reasons"]) or "- (no specific reasons -
    low risk)"

    prompt = PROMPT_TEMPLATE.format(
        risk_level=transaction["risk_level"],
        risk_score=transaction["risk_score"],
        amount=transaction["amount"],
        category=transaction["category"],
        country=transaction["country"],
        account_id=transaction["account_id"],
        reasons_bullets=reasons_bullets,
    )

    response = ollama.chat(
        model=OLLAMA_MODEL,
        messages=[{"role": "user", "content": prompt}],
        options={"temperature": 0.2, "num_predict": 220},
    )

```

```
return response["message"]["content"].strip()

if __name__ == "__main__":
```

9.2 Why this matters beyond hallucination

Flagging a transaction is a decision with consequences: a declined card, a frozen account, a customer stranded abroad. In many jurisdictions an automated decision of that kind carries a right to an explanation.

A GENERATED EXPLANATION IS NOT AN EXPLANATION

If the LLM produced the reasoning, the stated reason would be a plausible story about a decision rather than the decision's actual cause. The system would flag for reason A and tell the customer reason B, with nothing detecting the divergence.

Because the reasons are produced by the same code that produced the score, the explanation is *the* reason by construction. That property is worth more than any amount of fluency, and it is why **Project 11** takes the same approach for adverse-action notices, where the legal requirement is explicit.

CHAPTER 10

Worked Examples

Scoring three real transactions by hand, and the arithmetic of the recall ceiling.

10.1 Example 1 - Scoring a stolen-card transaction

An account with a typical spend of \$84.20, standard deviation \$29.50, home country IN, primary device D-4471. A transaction arrives: \$2,847.19, country RO, device D-9903, and it is the second from this account in the last minute.

12.1 Scoring a stolen-card-spree transaction

Rule	Test	Fires?	Points
1 - velocity	recent_count 2 >= 4	no	0
2 - amount	$z = (2847.19 - 84.20) / 29.50 = 93.7 > 6$	yes	+40
3 - country	RO <= IN	yes	+25
4 - device	D-9903 <= D-4471	yes	+20
		total	85 - High

85 matches the score the warehouse actually holds for twelve of the fifteen stolen-card events. The other three reached 100, where the velocity rule fired as well and the score was capped.

THREE INDEPENDENT SIGNALS AGREEING IS WHAT MAKES THIS EASY

Amount, geography and device all point the same way. Any two of them alone would still clear the 60-point High threshold. This scenario is caught at 100% recall because it is *over-determined* - which is exactly why the card-testing scenario, with only one signal available, is the interesting case.

10.2 Example 2 - Scoring a card-testing transaction

The same account. A transaction arrives: \$3.41, country IN, device D-4471, fifth in the last minute.

12.2 Scoring a card-testing transaction

Rule	Test	Fires?	Points
1 - velocity	recent_count 5 ≥ 4	yes	+35
2 - amount	$z = (3.41 - 84.20) / 29.50 = -2.7$	no	0
3 - country	IN = IN	no	0
4 - device	D-4471 = D-4471	no	0
		total	35 - Medium

THE Z-SCORE IS NEGATIVE, AND THAT IS THE WHOLE PROBLEM

Card testing uses *tiny* amounts - \$1 to \$5 - precisely because a small charge is less likely to be noticed.

The amount rule tests only $z > 3$, so a transaction far below normal scores nothing.

The attacker also stays in the home country on the usual device, because at this stage they are only checking whether the card number works. **Three of the four rules are structurally unavailable**, and every card-testing event in the warehouse scores exactly 0 or exactly 35 - never anything else.

10.3 Example 3 - The recall ceiling, derived

The velocity rule needs $\text{recent count} \geq 4$. Within a burst the count is 1, 2, 3, 4, 5, ... so the rule cannot fire on the first three events. For a burst of b events with a threshold of k :

$$\text{missed} = \min(b, k - 1), \quad \text{recall}_{\text{burst}} = \frac{b - (k - 1)}{b} = 1 - \frac{k - 1}{b}$$

The three card-testing bursts in this run had 7, 8 and 8 events. With $k = 4$:

12.3 The card-testing recall, predicted from the rule alone

Burst	Events	Missed	Caught	Recall
1	7	3	4	57.1%
2	8	3	5	62.5%
3	8	3	5	62.5%
Total	23	9	14	60.9%

60.9% WAS PREDICTABLE BEFORE RUNNING ANYTHING

The reported catch rate for card testing is 60.9%. The formula above gives $14/23 = 60.9\%$ from nothing but the velocity threshold and the burst sizes - no model, no data, no run.

A number you can derive from the design is a number you understand. It also means the figure is not evidence about tuning: it is a property of the rule, and no amount of adjusting the other three rules would move it.

10.4 Example 4 - Overall recall is at its structural maximum

44 fraud events, 35 caught, 9 missed - and all nine are the warm-up events of the three card-testing bursts.

$$\text{recall} = \frac{35}{44} = 0.795$$

The other two scenarios were caught at 100%. So of the 44 fraud events, exactly 9 were structurally undetectable by these four rules, and the detector caught **every single event it was capable of catching**.

WHICH REFRAMES WHAT 79.5% MEANS

Read as a tuning result, 79.5% recall invites the question 'why not higher?' and the answer would be to fiddle with thresholds.

Read correctly, it says: **this rule set has a ceiling of 79.5% on this data, and it is at the ceiling**. Improving it does not mean tuning - it means adding a signal the current rules do not have. Chapter 7 covers what that signal would be.

CHAPTER 11

Running the Project, Step by Step

Every terminal command needed to go from a fresh clone to a finished run, what each one does, and what to do when one of them fails.

11.1 Prerequisites

You need Python 3.11 or newer and Git. Verify both before starting - most reported 'the project does not work' turns out to be an environment that was never activated.

Check your toolchain

```
python --version
git --version
```

THIS PROJECT HAS ONE OPTIONAL LOCAL-LLM STAGE

Install Ollama and pull the model if you want the generative feature to run. It is optional: the project completes without it, and the deterministic control arm still produces results.

Optional - only for the LLM stage

```
ollama pull qwen2.5:0.5b
ollama list
```

ONE STAGE NEEDS WSL ON WINDOWS

The tooling this stage uses refuses to run on Windows directly. The chapter marks that command clearly and the project completes without it.

11.2 Step 1 - Clone and enter the repository

Get the code

```
git clone https://github.com/Nikhil201716/06-Realtime-Transaction-Streaming-Pipeline.git
cd 06-Realtime-Transaction-Streaming-Pipeline
```

11.3 Step 2 - Create an isolated environment

A virtual environment keeps this project's dependencies from colliding with anything else on your machine.

Create and activate

```
python -m venv .venv

# Windows (PowerShell)
.venv\Scripts\Activate.ps1

# macOS / Linux
source .venv/bin/activate
```

IF POWERSHELL REFUSES TO RUN THE ACTIVATION SCRIPT

Windows blocks unsigned scripts by default. Allow them for your own user account only:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

11.4 Step 3 - Install dependencies

Install

```
pip install -r requirements.txt
```

11.5 Step 4 - Run the pipeline, stage by stage

Each command below produces the input the next one consumes. Run them in this order:

The full pipeline in order

```
redis-server --daemonize yes
redis-cli ping
python scripts/generate_reference_data.py
python scripts/producer.py --duration 180 --rate 2.5
python pipeline/consumer.py --duration 200
python scripts/evaluate_detection.py
```

What each stage does:

1. `yes` - starts Redis - on Windows this must happen inside WSL, see the note below
2. `ping` - confirms Redis is reachable; it should answer PONG before you go further
3. `scripts/generate_reference_data.py` - builds the seeded account master with each account's home country, primary device and historical spend profile
4. `2.5` - streams normal traffic plus the three injected fraud scenarios into Redis
5. `200` - reads the stream through a consumer group, maintains the velocity window, scores every event, records latency and writes to DuckDB

6. `scripts/evaluate_detection.py` - joins the scored events against the injected labels and writes the confusion matrix, per-scenario catch rates and latency percentiles

EXPECTED OUTPUT

Redis does not run natively on Windows. This project was developed against Redis 8.10.0 built from source in WSL; Memurai is a Windows alternative if you prefer. Everything except the stream itself - the evaluation, the dashboard, the warehouse queries - runs on Windows directly, because those only read the DuckDB file the consumer already wrote. Expect precision near 97.2%, recall near 79.5%, and all nine misses to be card-testing events. If card testing catches more than about 61%, check whether the velocity threshold has been changed.

11.6 Run producer and consumer together

Either order works - the consumer group reads from the beginning of the stream, so starting the consumer after the producer has finished still processes every event.

```
python scripts/producer.py --duration 180 --rate 2.5 &
python pipeline/consumer.py --duration 200
```

11.7 Generate an explanation for a flagged transaction

Turns the reason strings the rules computed into an analyst-readable narrative using the local model. Needs Ollama; every other stage does not.

```
python pipeline/explain.py
```

11.8 Inspect the stream directly

Shows the stream length and, for each consumer group, its pending count - the number of delivered but unacknowledged messages.

```
redis-cli XINFO STREAM transactions:raw
redis-cli XINFO GROUPS transactions:raw
```

11.9 Inspect a velocity window

The sliding-window sorted set for one account, with timestamps as scores. Expect it to be empty unless a run is in progress, since the key expires.

```
redis-cli ZRANGE velocity:ACC300960 0 -1 WITHSCORES
```

11.10 Query the scored warehouse

A quick check that scoring produced the expected distribution.

```
python -c "import duckdb; print(duckdb.connect('database/streaming.duckdb',
read_only=True).execute('SELECT risk_level, COUNT(*) FROM transaction_scores GROUP BY 1').fetchall())"
```

11.11 Launch the operations console

Opens on <http://localhost:8501>, reading the same DuckDB file the consumer writes.

```
streamlit run dashboard/streamlit_app.py
```

11.12 Command reference card

Every command from this chapter in one block, for when you already know what you want.

Complete command reference

```
# --- setup (once) ---
git clone https://github.com/Nikhil201716/06-Realtime-Transaction-Streaming-Pipeline.git
cd 06-Realtime-Transaction-Streaming-Pipeline
python -m venv .venv
.venv\Scripts\Activate.ps1          # Windows
source .venv/bin/activate           # macOS / Linux
pip install -r requirements.txt

# --- run ---
redis-server --daemonize yes
redis-cli ping
python scripts/generate_reference_data.py
python scripts/producer.py --duration 180 --rate 2.5
python pipeline/consumer.py --duration 200
python scripts/evaluate_detection.py

# --- extras ---
python scripts/producer.py --duration 180 --rate 2.5 &
python pipeline/consumer.py --duration 200
python pipeline/explain.py
redis-cli XINFO STREAM transactions:raw
redis-cli XINFO GROUPS transactions:raw
redis-cli ZRANGE velocity:ACC300960 0 -1 WITHSCORES
python -c "import duckdb; print(duckdb.connect('database/streaming.duckdb',
read_only=True).execute('SELECT risk_level, COUNT(*) FROM transaction_scores GROUP BY 1').fetchall())"
streamlit run dashboard/streamlit_app.py

# --- finish ---
deactivate
```

11.13 Troubleshooting

Common problems and their fixes

Symptom	Cause	Fix
<code>ModuleNotFoundError</code>	environment not activated	re-run the activate command from Step 2
<code>FileNotFoundError</code> on a data file	an earlier stage was skipped	run the stages in the documented order
Numbers differ from this book	a seed or parameter was changed	check the constants at the top of the generator
<code>command not found</code>	dependency missing from this env	<code>pip install -r requirements.txt</code> with the env active
Permission denied writing output	directory created by another user or process	delete the generated output directory and re-run

CHAPTER 12

The Live Dashboard

What an operations console for a streaming system needs to show, and why the reasons column is the important one.

12.1 Reading a stream that has already been scored

`dashboard/streamlit_app.py` reads the DuckDB table the consumer writes. It does not participate in the stream, which means it can be opened, closed and reloaded without any risk to the pipeline - the same separation as the dashboards in Projects 4 and 5.

Four things an analyst needs: the flagged queue ordered by risk score, the latency distribution, the score distribution across all events, and - most importantly - **the reasons for each flag**.

Launch the console

```
streamlit run dashboard/streamlit_app.py
```

12.2 Why the reasons column carries the product

A queue of flagged transactions sorted by score is a list of numbers. An analyst cannot act on a number - they have to decide whether to call the customer, and for that they need to know what the system saw.

THE DASHBOARD SHOWS WHERE THE DETECTOR IS WEAKEST

Sorted by score, the flagged queue has a clear structure: everything at 60 and above is fraud, and the 35-point band contains fourteen true positives and the single false positive.

That band is where an analyst's judgement is actually needed, and the reasons column is what makes the judgement possible - although, as Chapter 7 shows, in this case the reason strings for the true and false positives are *identical*, so even a human has nothing to go on without pulling more context.

CHAPTER 13

Results

Everything measured, in one place, with the structural findings stated next to the headline metrics.

13.1 The numbers

13.1 Every measured figure

Metric	Value
Events streamed and scored	415
Fraud events injected	44 across 3 scenarios
Precision	97.2% (35 of 36 flags correct)
Recall	79.5% (35 of 44 caught)
F1	0.875
account_takeover catch rate	100% (6/6)
stolen_card_sprees catch rate	100% (15/15)
card_testing catch rate	60.9% (14/23)
Latency average	6.2 ms
Latency p95	21.1 ms
Latency max	30.4 ms

13.2 The four findings worth defending

1. Recall is at its structural ceiling, not below it

All nine misses are warm-up events of card-testing bursts, where fewer than four transactions had yet been seen. The detector caught 100% of what it was capable of catching. Improving it requires a new signal, not new thresholds.

2. The catch rate for card testing was predictable without running anything

$1 - (k - 1)/b$ over bursts of 7, 8 and 8 gives $14/23 = 60.9\%$, the measured value exactly. A number derivable from the design is a number that is understood.

3. The single false positive is indistinguishable from 14 true positives

Same score, same reason, same rule. The rule set cannot separate a bursting legitimate account from card testing at all; precision of 97.2% reflects how rarely normal accounts burst, not any discriminating power.

4. The system is five points from blindness to one attack class

Card testing scores exactly 35 against a Medium threshold of 30. Raising alerting to High would take its recall to zero while making precision look better - a change that would read as an improvement on every summary metric.

13.3 The honest tooling note

Redis does not run natively on Windows and Memurai was not installed in this environment, so Redis 8.10.0 was built from source in WSL. That is stated in the project's setup documentation rather than glossed.

WHY THE DEPENDENCY IS WORTH FLAGGING

The portfolio's standing constraint is that anyone should be able to run these projects for free, indefinitely, on modest hardware. This project does not fully meet it: a Windows user needs WSL and a source build before anything runs.

Project 14 revisits the same problem and resolves it - Redis becomes an optional transport with a self-contained embedded append-only log as the fallback, so the project runs anywhere while still demonstrating the real thing when Redis is available. Naming the constraint here is what made that design decision obvious later.

CHAPTER 14

Exercises, with Solutions

Problems requiring the chapters, with worked answers.

14.1 Exercises

1. A velocity rule uses threshold k and an attacker sends bursts of b events. Derive the recall on that scenario and evaluate it for $k = 4, b = 6$.
2. The consumer acknowledges after writing to the warehouse. Describe what happens if it crashes between the write and the XACK, and give the fix.
3. Why does `ZADD` come before `ZREMRANGEBYSCORE` in the pipeline, and what would break if the order were swapped?
4. The amount rule tests $z > 3$. Explain why this makes card testing invisible to it, and propose a rule that would catch the first event of a card-testing burst.
5. A colleague raises the alerting threshold from Medium to High to reduce alert volume. Predict the effect on precision, recall, and each scenario's catch rate.
6. The single false positive scored 35 with the reason "8 transactions in 60 seconds". What extra information would let a detector separate it from card testing?
7. Why is accuracy a useless metric here? Compute it for a detector that flags nothing.
8. Give the 95% Wilson interval for a precision estimate of 35/36 and say what it implies about the reported 97.2%.

14.2 Solutions

1. The warm-up formula

The count within a burst is 1, 2, ..., b , so the rule fires from the k th event onward, missing $\min(b, k - 1)$. Recall is $(b - k + 1)/b = 1 - (k - 1)/b$. For $k = 4, b = 6$: $1 - 3/6 = 50\%$. Note the ceiling gets *worse* as bursts get shorter - an attacker who splits one burst of 8 into two of 4 drops the detector's recall from 62.5% to 25%.

2. The crash between write and ack

The message stays in the pending entries list, so on restart it is redelivered. The consumer scores it again, writes it again, and - the subtler problem - `ZADD`s it into the velocity window again, inflating that account's count. The fix has two parts: upsert the warehouse row on `transaction_id` so a re-write is idempotent, and make the window member `transaction_id`-keyed rather than timestamp-keyed so re-adding the same event replaces rather than duplicates it.

3. The pipeline order

Adding before evicting means the count returned by `ZCARD` includes the current event, which is what `recent_count >= 4` assumes. Swap them and every count is one lower, so the rule fires one transaction later - extending the warm-up blind spot from three events to four and dropping card-testing recall from 60.9% to about 47.8%.

4. The upper-tail problem

Card testing uses \$1-\$5 against an average of \$84, so Z is strongly *negative* and a test of $Z > 3$ never fires. A rule catching the first event would test the lower tail - for example $Z < -2$ combined with a card-not-present category - since a charge far below an account's norm at an online merchant is the signature of a probe. This is new information rather than new tuning, which is the point of Chapter 7.

5. Raising the threshold to High

Card testing scores exactly 35, so all 14 of its true positives disappear - catch rate goes to **0%**. The single false positive also scored 35, so it disappears too. `account_takeover` (60, 95) and `stolen_card_spree` (85, 100) are unaffected at 100%. New totals: TP 21, FP 0, FN 23. **Precision rises to 100%, recall falls to 47.7%, F1 falls from 0.875 to 0.646.** Precision and alert volume both improve, which is exactly why this change is dangerous - it looks like a win on the metrics a team usually watches.

6. Separating the false positive

Nothing in the current feature set can - the two are score- and reason-identical. What would help: the *amounts* in the burst (card testing is uniformly tiny; a legitimate burst is varied), the *merchant spread* (a probe hits several merchants, a real burst usually one), and the account's own history of bursting (some customers always do). All three are additional signal, not better tuning.

7. Accuracy on an imbalanced problem

A detector flagging nothing gets TP 0, FP 0, FN 44, TN 371, so accuracy is $371/415 = 89.4\%$. It catches no fraud whatsoever. Accuracy is dominated by the majority class and therefore measures the base rate rather than the detector - and at a realistic fraud rate of 0.1% the same useless detector would score 99.9%.

8. The Wilson interval on precision

For 35/36 the 95% Wilson interval runs from roughly 86% to 99.5%. The reported 97.2% is a point estimate resting on a single false positive: one more would have made it 94.6%, and one fewer 100%. The honest reading is that precision is *high* and this run cannot pin it down more tightly than that - the same sample-size discipline that forced the regional retraction in Project 3.

CHAPTER 15

Appendix

Repository map, glossary, and reproducibility notes.

15.1 A - Repository map

Where everything lives

Path	Purpose
scripts/generate_reference_data.py	the seeded account master and spend profiles
scripts/producer.py	the live feed and the three injected fraud scenarios
scripts/evaluate_detection.py	post-hoc grading against the injected labels
pipeline/redis_client.py	connection handling
pipeline/consumer.py	consumer group, sliding window, scoring, latency, write, ack
pipeline/scoring_rules.py	the four rules, isolated and unit-testable
pipeline/explain.py	the local-LLM narrative layer over computed reasons
dashboard/streamlit_app.py	the operations console
reports/detection_evaluation.json	the machine-readable results
reports/detection_evaluation.md	the human-readable summary
docs/architecture.md	design rationale
docs/wsl_redis_setup.md	the exact environment this project ran in

15.2 B - Glossary

Terms used in this book

Term	Meaning
Seeded generator	a synthetic data producer whose random draws are reproducible from a fixed starting value
Ground truth	the mechanism deliberately injected into synthetic data, used only to score an analysis - never to make a decision
Control arm / baseline	a simple alternative a new method must beat before it is worth its complexity
Point-in-time correctness	computing a feature using only information that existed at the moment being modelled
Leakage	information from the future or from the target entering a feature, inflating every offline metric
Reproducibility checksum	a hash of generated output, compared across a fresh process to prove determinism
Stream / log	an append-only sequence of entries that consumers read by position
Queue	a structure where consuming a message removes it
XADD / XREADGROUP / XACK	Redis Streams: append, read as a group member, acknowledge
Consumer group	a set of consumers sharing a stream, with server-tracked positions
Pending entries list	delivered but unacknowledged messages, visible for recovery
At-least-once	no message is lost; duplicates are possible
Exactly-once	at-least-once transport plus an idempotent sink
Sliding window	a count over the most recent time interval, maintained incrementally
Sorted set (ZSET)	a Redis structure ordering members by a numeric score
Pipelining	sending several Redis commands in one round-trip
Event time vs processing time	when it happened versus when it was seen
Velocity rule	flagging on the count of recent events from one entity
Warm-up blind spot	the first k-1 events of a burst, which a velocity rule cannot fire on
Additive scoring	independent rules contributing points to a total
Precision / recall / F1	flag correctness, fraud coverage, and their harmonic mean
Class imbalance	one outcome vastly outnumbering the other, making accuracy useless
Label leakage	the scorer seeing the ground truth it is being graded against

Term	Meaning
Percentile latency	p95 or p99 - the honest summary of a skewed distribution

15.3 C - Reproducibility

Every dataset in this project is produced by a seeded generator, so the whole pipeline can be rebuilt from nothing and should produce byte-identical output.

A FIXED SEED IS A CLAIM, NOT A GUARANTEE

Two projects in this portfolio shipped generators with fixed seeds that were still not reproducible across runs. The causes were `rng.choice(list(a_set))` and `abs(hash(some_string))` - both of which are perfectly stable *within* one process and vary between processes, because Python randomises string hashing per interpreter run.

Neither is visible from reading the code, from the seed, or from the output looking sensible. Both were caught only by hashing the generated files and comparing across a *fresh* process. The defences used throughout this portfolio are to sort any collection before it feeds a random draw, and to verify by checksum in a new process rather than trusting the seed.

15.4 D - Where this leads next

This project introduces streaming and real-time decisioning. The thread continues: **Project 7** puts a model behind a serving API and adds drift monitoring, so the scoring logic can change without redeploying rules. **Project 9** replaces per-account rules with graph analysis and finds fraud *rings* that no single-account rule can see. **Project 12** builds the learned model that discovers the interactions an additive rule set cannot express. **Project 14** takes the timing problem seriously - event time, watermarks, late data, exactly-once sinks - and also resolves this project's Redis dependency with an embedded fallback. **Project 15** shows what happens when a fraud detector meets an adversary who adapts: 95.5% recall collapsing to 18.2%.

The three most valuable improvements to *this* project, in order: add a lower-tail amount rule so the first event of a card-testing burst is visible, which is the only way past the recall ceiling; make the consumer idempotent on `transaction_id` so a redelivery cannot double-count the velocity window; and load-test until the consumer falls behind, since the latency figures currently describe only the regime where the system has slack.